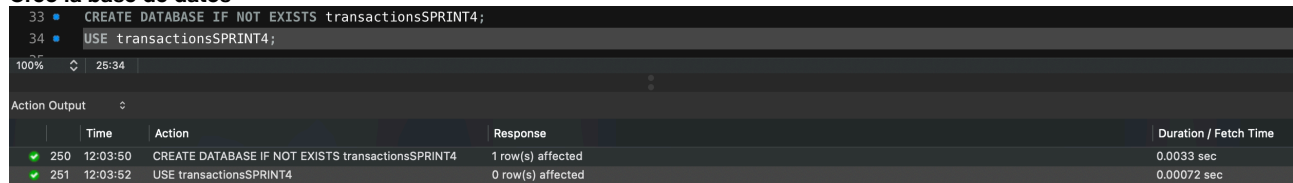


Emilio Tornos Iniesta - Tasca S4.01. Creació de Base de Dades

NIVELL 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Creo la base de datos

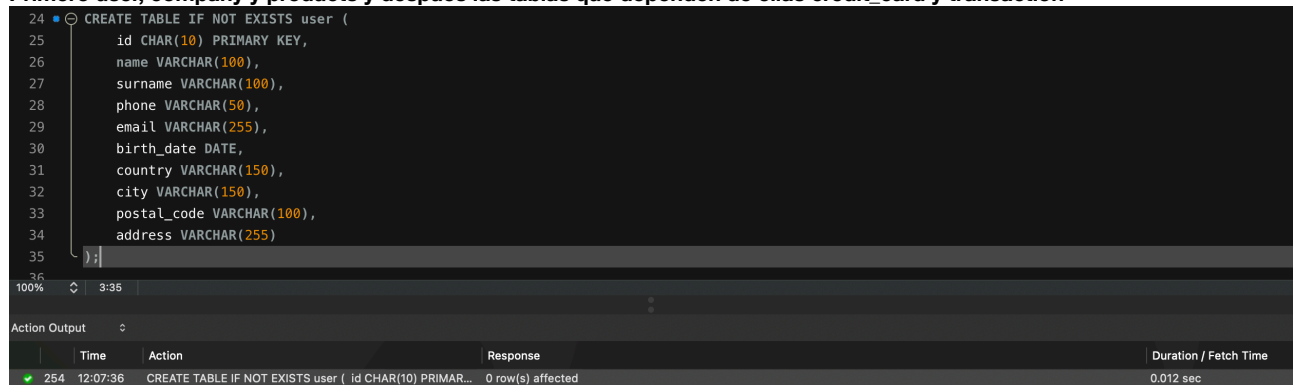


```
33 CREATE DATABASE IF NOT EXISTS transactionsSPRINT4;
34 USE transactionsSPRINT4;
```

	Time	Action	Response	Duration / Fetch Time
250	12:03:50	CREATE DATABASE IF NOT EXISTS transactionsSPRINT4	1 row(s) affected	0.0033 sec
251	12:03:52	USE transactionsSPRINT4	0 row(s) affected	0.00072 sec

- Creo la base de datos **transactionsSPRINT4** y la defino como la opción a usar por defecto.

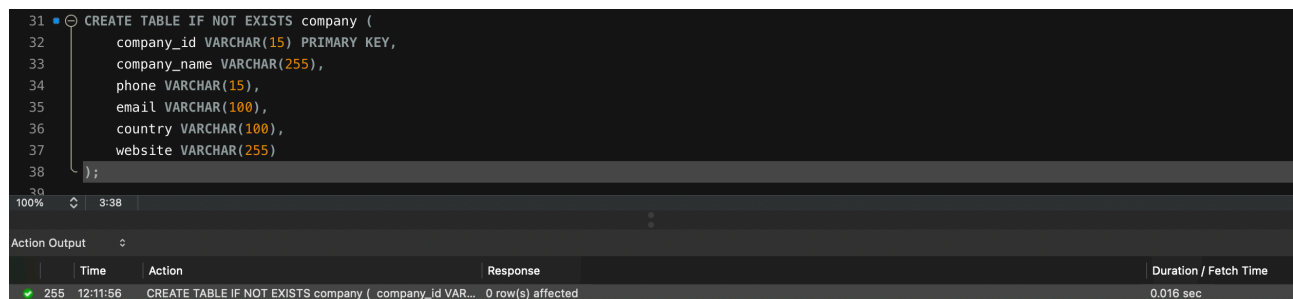
Creo las tablas asegurando que se creen en el orden correcto, las tablas padres deben existir antes que las tablas hijas. Primero **user**, **company** y **products** y después las tablas que dependen de ellas **credit_card** y **transaction**



```
24 CREATE TABLE IF NOT EXISTS user (
25     id CHAR(10) PRIMARY KEY,
26     name VARCHAR(100),
27     surname VARCHAR(100),
28     phone VARCHAR(50),
29     email VARCHAR(255),
30     birth_date DATE,
31     country VARCHAR(150),
32     city VARCHAR(150),
33     postal_code VARCHAR(100),
34     address VARCHAR(255)
35 );
```

	Time	Action	Response	Duration / Fetch Time
254	12:07:36	CREATE TABLE IF NOT EXISTS user (id CHAR(10) PRIMAR...	0 row(s) affected	0.012 sec

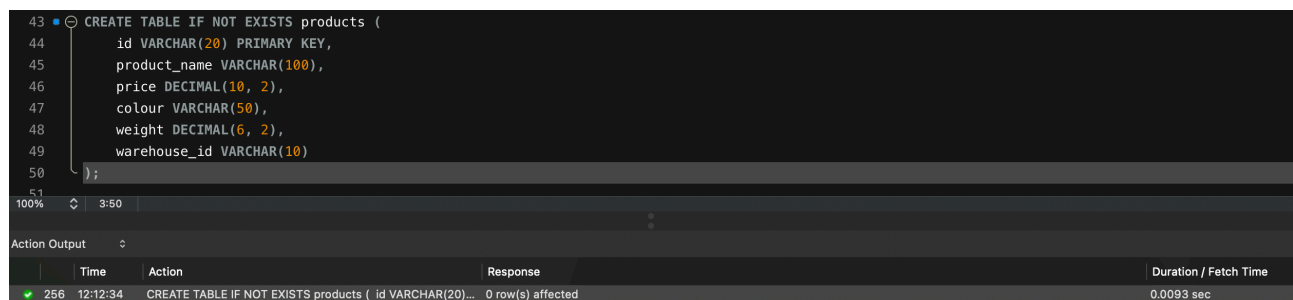
- Creo la tabla **user**
- Para **ID** escojo **VARCHAR(20)** para mayor flexibilidad y seguridad.
- Para **EMAIL** escojo **VARCHAR(255)** porque los estándares técnicos para emails permiten hasta 254 caracteres.
- Para **BIRTH_DATE** escojo **DATE** para permitir validación, ordenación y cálculo.
- Para **PHONE** escojo **VARCHAR(50)** que considero que ya es suficiente para cubrir todas las posibles casuísticas en formatos internacionales.



```
31 CREATE TABLE IF NOT EXISTS company (
32     company_id VARCHAR(15) PRIMARY KEY,
33     company_name VARCHAR(255),
34     phone VARCHAR(15),
35     email VARCHAR(100),
36     country VARCHAR(100),
37     website VARCHAR(255)
38 );
```

	Time	Action	Response	Duration / Fetch Time
255	12:11:56	CREATE TABLE IF NOT EXISTS company (company_id VAR...	0 row(s) affected	0.016 sec

- Creo la tabla **company**



```
43 CREATE TABLE IF NOT EXISTS products (
44     id VARCHAR(20) PRIMARY KEY,
45     product_name VARCHAR(100),
46     price DECIMAL(10, 2),
47     colour VARCHAR(50),
48     weight DECIMAL(6, 2),
49     warehouse_id VARCHAR(10)
50 );
```

	Time	Action	Response	Duration / Fetch Time
256	12:12:34	CREATE TABLE IF NOT EXISTS products (id VARCHAR(20)...	0 row(s) affected	0.0093 sec

- Creo la tabla **products**
- Para **ID** escojo **VARCHAR(20)** para mayor flexibilidad y seguridad.
- En **product_name**, como el nombre de un producto es **texto**, uso un **VARCHAR** adecuado.
- Para **price** escojo **DECIMAL(10, 2)**, 10 dígitos totales y 2 decimales por ser una cantidad monetaria.
- En **weight**, siendo el peso una cantidad que puede contener decimales, escojo **DECIMAL(6, 2)** en vez de **FLOAT** para mejor precisión.

```
58 CREATE TABLE IF NOT EXISTS credit_card (
59     id VARCHAR(20) PRIMARY KEY,
60     user_id CHAR(10),
61     iban CHAR(34),
62     pan VARCHAR(19),
63     pin CHAR(4),
64     cvv CHAR(4),
65     track1 VARCHAR(100),
66     track2 VARCHAR(100),
67     expiring_date CHAR(10)
68 );
```

Time	Action	Response	Duration / Fetch Time
12:14:25	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(20) PRIMARY KEY, user_id CHAR(10), iban CHAR(34), pan VARCHAR(19), pin CHAR(4), cvv CHAR(4), track1 VARCHAR(100), track2 VARCHAR(100), expiring_date CHAR(10));	0 row(s) affected	0.015 sec

- Creo la tabla **credit_card**
- Siguiendo la sintonía de la longitud de los ID en los datos a introducir, VARCHAR(20) es más flexible.
- El **user_id** VARCHAR(20) coincidiendo con el user.id
- El **iban** más largo conocido es de 34 caracteres de longitud, por eso escojo CHAR(34)
- El **pan** (número de la tarjeta) generalmente está entre los 16 y los 19 caracteres de longitud, por eso escojo VARCHAR(19)
- **No usamos INT** en pin o cvv para **preservar los 0** y conservar el formato exacto, por eso escojo **CHAR**, y 4 de longitud porque he visto que también **existen cvv de esa longitud**, además usando **CHAR** le indicamos a la base de datos que **no son valores numéricos para cálculo**, lo que mejora la seguridad y la velocidad de indexación.
- En el caso de **expiring_date** utilizo **CHAR(10) provisionalmente** para evitar problemas en la carga, **posteriormente definiré el tipo de dato como DATE**.

```
83 CREATE TABLE IF NOT EXISTS transaction (
84     id VARCHAR(255) PRIMARY KEY,
85     card_id VARCHAR(20),
86     business_id VARCHAR(15),
87     timestamp TIMESTAMP,
88     amount DECIMAL(10, 2),
89     declined TINYINT,
90     product_ids VARCHAR(255),
91     user_id CHAR(10),
92     lat DECIMAL(10, 7),
93     longitude DECIMAL(10, 7)
94 );
```

Time	Action	Response	Duration / Fetch Time
12:18:15	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(20), business_id VARCHAR(15), timestamp TIMESTAMP, amount DECIMAL(10, 2), declined TINYINT, product_ids VARCHAR(255), user_id CHAR(10), lat DECIMAL(10, 7), longitude DECIMAL(10, 7));	0 row(s) affected	0.020 sec

- Para **card_id** escojo **VARCHAR(20)** coincidiendo con **credit_card.id**
- Para **business_id** escojo **VARCHAR(15)** coincidiendo con **company.company_id**
- Para **timestamp** escojo **TIMESTAMP** que es el tipo estándar para la hora exacta de la transacción.
- Para **amount** escojo **DECIMAL(10, 2)** que es el tipo estándar para cantidades monetarias.
- Para **declined** escojo **TINYINT** que es la representación estándar de BOOLEAN en MySQL, que se necesita almacenar valores booleanos (verdadero o falso).
- En el caso de **product_ids** utilizo **VARCHAR(255)** porque se almacenan múltiples IDs como en una lista.
- Para **user_id** escojo **CHAR(10)** coincidiendo con **users.id**
- Para **lat** y **longitude**, escojo **DECIMAL(10, 7)** en vez de **FLOAT** porque es más preciso para coordenadas geográficas, 10 dígitos totales, 7 decimales es un estándar común.

Carga los datos de las tablas

```
105 LOAD DATA INFILE 'american_users.csv'
106 INTO TABLE user;
```

Time	Action	Response	Duration / Fetch Time
10:03:18	LOAD DATA INFILE 'american_users.csv' INTO TABLE user;	Error Code: 1290. The MySQL server is running with the --secure-file-priv option so it cannot execute this statement	0.00045 sec

- Al querer cargar el **csv** a través de **LOAD DATA INFILE** me devuelve un error que básicamente me dice que por un tema de configuración de seguridad del servidor en mi mac, solo puedo cargar archivos que estén en una ruta específica y segura.

111 SHOW VARIABLES LIKE 'secure_file_priv';
112

100% 40:111

Result Grid Filter Rows: Search Export:

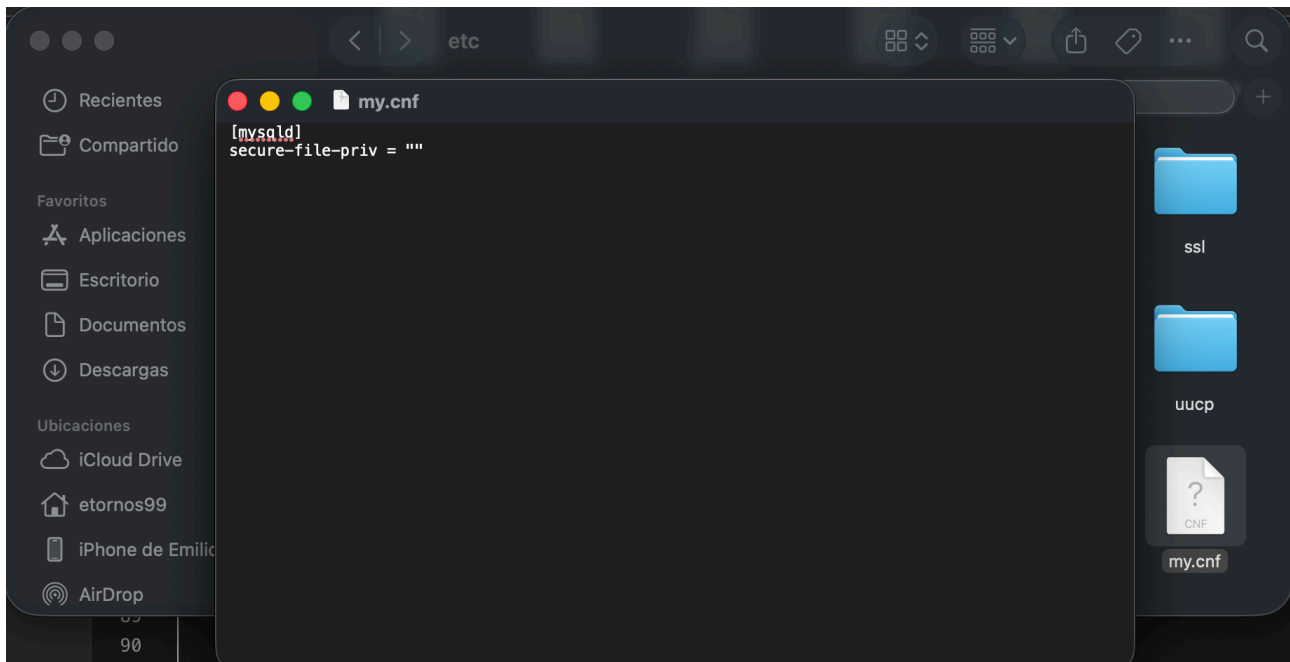
Variable_name	Value
secure_file_priv	NULL

Result 1 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
11	09:57:36	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0060 sec / 0.00001...

- Ejecutando este comando veo que el servidor en vez de devolverme una tabla con el valor de la ruta me devuelve NULL, eso significa que no tengo ninguna ruta segura designada.
- La funcionalidad de carga de archivos externos está **completamente deshabilitada** en el servidor MySQL por defecto.
- No existe el archivo de configuración (my.cnf) en las rutas esperadas, lo que impide cambiar la configuración directamente.



Para habilitar la carga, He tenido que forzar al servidor a aceptar un nuevo archivo de configuración:

- **Creando el archivo my.cnf:** Creo manualmente el archivo my.cnf en un editor de texto con el contenido [mysqld] secure-file-priv = "" y lo coloqué en la ubicación /etc/, una de las rutas que el servidor busca por defecto.
- 1. **[mysqld]** - Indica que la configuración dentro de esta sección se aplica al proceso principal del servidor MySQL.
- 2. **secure-file-priv** - Esta es una variable de seguridad que controla qué carpetas del sistema operativo, puede el servidor MySQL leer o escribir cuando se ejecutan comandos como LOAD DATA INFILE o SELECT INTO OUTFILE.
- 3. **"" (Cadena vacía):** Al establecer su valor como una cadena vacía, se **deshabilita** por completo la restricción de seguridad. Esto permite que el servidor MySQL lea los archivos CSV desde **cualquier ubicación** del sistema operativo.
- **Anulación de la Restricción:** La línea secure-file-priv = "" anula la restricción de seguridad, permitiendo la carga desde cualquier ubicación.
- **Por último:** Reinicio el servicio de MySQL para que el servidor lea el nuevo archivo de configuración.

28 SHOW VARIABLES LIKE 'secure_file_priv';

100% 40:28

Result Grid Filter Rows: Search Export:

Variable_name	Value
secure_file_priv	

Result 1 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
1	10:39:41	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0045 sec / 0.00001...

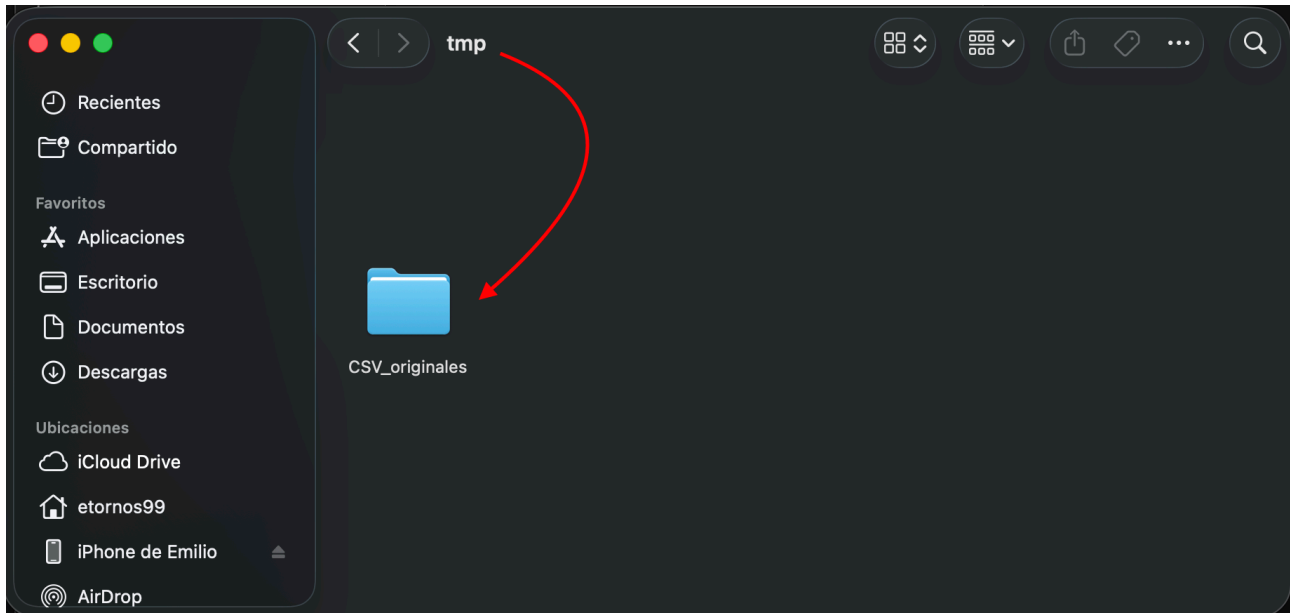
- Vuelvo a ejecutar el código para que el servidor me devuelva una tabla con el valor de la ruta y ahora me aparece **vacía** en vez de **NULL**, eso significa que:
- He creado el archivo my.cnf correctamente.
- La línea secure-file-priv = "" fue leída por el servidor.
- La restricción de seguridad se ha **levantado**.
- En principio ahora podré usar LOAD DATA INFILE para cargar los archivos CSV de cualquier ubicación de mi ordenador.

```
105 LOAD DATA INFILE '/Users/etornos99/Documents/IT_ACADEMY_ESPECIALIZACIÓN/SPRINT4/CSV_originales/American_users.csv'
106 INTO TABLE user;

Action Output

Time Response Duration / Fetch Time
13 11:02:36 Li Error Code: 13. Can't get stat of '/Users/etornos99/Documents/IT_ACADEMY_ESPECIALIZACIÓN/SPRINT4/CSV_originales/American_users.csv' (OS errno 13 - Permission denied) 0.0017 sec
```

- Una vez levantada la restricción de MySQL, el sistema me devuelve el **Error Code 13: Permission denied**.
- Este no es un problema de la base de datos, sino del sistema operativo (macOS), que por defecto restringe el acceso de los procesos del servidor a los archivos dentro de la carpeta personal.



- Incluso después de intentar modificar los permisos de lectura, la restricción del sistema persiste.
- **Solución Final:** Para resolver el problema de permisos de manera definitiva, muevo los archivos CSV a la carpeta temporal universal **/tmp/** del sistema operativo, que tiene permisos de lectura abiertos para todos los procesos, incluido el servidor MySQL.

```
125 LOAD DATA INFILE '/private/tmp/CSV_originales/American_users.csv'
126 INTO TABLE user
127 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas
128 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
129 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
130 (
131   @id, @name, @surname, @phone, @email, @birth_date_str,
132   @country, @city, @postal_code, @address
133 )
134 SET
135   id = @id,
136   name = @name,
137   surname = @surname,
138   phone = @phone,
139   email = @email,
140   birth_date = STR_TO_DATE(@birth_date_str, '%b %d, %Y'),
141   country = @country,
142   city = @city,
143   postal_code = @postal_code,
144   address = @address;

Action Output

Time Action Response Duration / Fetch Time
260 12:23:13 LOAD DATA INFILE '/private/tmp/CSV_originales/American_... Error Code: 1411. Incorrect datetime value: 'Nov 17' for function str_to_date 0.0070 sec
```

- Intento cargar los datos del csv **american_users.csv** en la tabla **user**.
- Me da error porque el CSV está usando comillas dobles " para encerrar los valores de los campos.


```

148 LOAD DATA INFILE '/private/tmp/CSV_originales/American_users.csv'
149 INTO TABLE user
150 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas
151 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
152 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
153 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
154 (
155     @id, @name, @surname, @phone, @email, @birth_date_str,
156     @country, @city, @postal_code, @address
157 )
158 SET
159     id = @id,
160     name = @name,
161     surname = @surname,
162     phone = @phone,
163     email = @email,
164     birth_date = STR_TO_DATE(@birth_date_str, '%b %d, %Y'),
165     country = @country,
166     city = @city,
167     postal_code = @postal_code,
168     address = @address;

```

100% 24:168

	Time	Action	Response	Duration / Fetch Time
✓ 261	12:25:56	LOAD DATA INFILE '/private/tmp/CSV_originales/American_...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.024 sec

- Cargó los datos del **CSV American_users** en la tabla **user** correctamente:
- Añadiendo **ENCLOSED BY ""** para indicar que debe ignorar las dobles comillas " al leer los campos.
- Y utilizando **STR_TO_DATE()** para **convertir la cadena de texto** en una **fecha**, en este caso con;
 - **%b**: Que se utiliza para un nombre abreviado del mes (por ejemplo, Ene)
 - **%d**: Que se utiliza para el día del mes (01 a 31), añadiendo la , que tiene a su derecha
 - **%Y**: Que se utiliza para el año con 4 dígitos

```

176 LOAD DATA INFILE '/private/tmp/CSV_originales/European_users.csv'
177 INTO TABLE user
178 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas
179 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
180 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
181 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
182 (
183     @id, @name, @surname, @phone, @email, @birth_date_str,
184     @country, @city, @postal_code, @address
185 )
186 SET
187     id = @id,
188     name = @name,
189     surname = @surname,
190     phone = @phone,
191     email = @email,
192     birth_date = STR_TO_DATE(@birth_date_str, '%b %d, %Y'),
193     country = @country,
194     city = @city,
195     postal_code = @postal_code,
196     address = @address;

```

100% 24:196

	Time	Action	Response	Duration / Fetch Time
✓ 262	12:29:34	LOAD DATA INFILE '/private/tmp/CSV_originales/European_...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.053 sec

- Repito el proceso con los datos del **CSV european_users** en la tabla **user** para **unificarlo** todo en **una sola tabla**.

Ahora vuelvo a crear una tabla temporal que se llame **european_users** solamente para usarla de filtro en mi tabla **user**, y poder añadir una columna que se llame 'continent', con el valor 'europe' o 'america' según corresponda.

```
203 CREATE TABLE IF NOT EXISTS european_users (  
204     id CHAR(10) PRIMARY KEY,  
205     name VARCHAR(100),  
206     surname VARCHAR(100),  
207     phone VARCHAR(50),  
208     email VARCHAR(255),  
209     birth_date DATE,  
210     country VARCHAR(150),  
211     city VARCHAR(150),  
212     postal_code VARCHAR(100),  
213     address VARCHAR(255)  
214 );  
215  
100% 3:214  
Action Output  
Time Action Response Duration / Fetch Time  
263 12:32:03 CREATE TABLE IF NOT EXISTS european_users ( id CHAR(1... 0 row(s) affected 0.012 sec
```

- Uso una estructura idéntica a mi tabla **user** para crear la tabla **european_users**

```
218 LOAD DATA INFILE '/private/tmp/CSV_originales/European_users.csv'  
219 INTO TABLE european_users  
220 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas  
221 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.  
222 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea  
223 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)  
224 (  
225     @id, @name, @surname, @phone, @email, @birth_date_str,  
226     @country, @city, @postal_code, @address  
227 )  
228 SET  
229     id = @id,  
230     name = @name,  
231     surname = @surname,  
232     phone = @phone,  
233     email = @email,  
234     birth_date = STR_TO_DATE(@birth_date_str, '%b %d, %Y'),  
235     country = @country,  
236     city = @city,  
237     postal_code = @postal_code,  
238     address = @address;  
239  
100% 24:238  
Action Output  
Time Action Response Duration / Fetch Time  
264 12:32:56 LOAD DATA INFILE '/private/tmp/CSV_originales/European_... 3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0 0.052 sec
```

- Cargo los datos del **csv european_users** en esta nueva tabla **european_users**

```
242 SELECT european_users.id  
243 FROM european_users;  
244  
100% 21:243  
Result Grid  
id  
1000  
1001  
1002  
1003  
1004  
1005  
1006  
1007  
1008  
1009  
1010  
european_users 7 Apply Revert  
Form Editor  
Action Output  
Time Action Response Duration / Fetch Time  
266 12:34:18 SELECT european_users.id FROM european_users 3990 row(s) returned 0.0021 sec / 0.00081...
```

- Hago una **select** a la **nueva tabla temporal european_users** que acabo de crear para posteriormente añadir la nueva columna llamada **continent**.

```

242 SELECT european_users.id
243 FROM european_users;
244
245 #Hago una select a la nueva tabla temporal european_users que acabo de crear para posteriormente añadir la nueva columna llamada continent.
246
247 ALTER TABLE user
248 ADD COLUMN continent VARCHAR(50);
249
100% 34:248

```

	Time	Action	Response	Duration / Fetch Time
✓ 267	12:35:56	ALTER TABLE user ADD COLUMN continent VARCHAR(50)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.012 sec

- Creo la **nueva columna** en la tabla user llamada '**Continent**'

Completo los valores de la columna user.continent con europe o america según corresponda.

```

254 UPDATE user
255 SET user.continent = "Europe"
256 WHERE user.id IN (
257     SELECT european_users.id
258     FROM european_users);
259
100% 23:258

```

	Time	Action	Response	Duration / Fetch Time
✓ 268	12:37:22	UPDATE user SET user.continent = "Europe" WHERE user.id...	3990 row(s) affected Rows matched: 3990 Changed: 3990 Warnings: 0	0.081 sec

- En este caso utilizo **UPDATE** para **añadir Europe** en la columna user.continent de la tabla user, si los user.id son el mismo que los de european_users.id

```

263 UPDATE user
264 SET user.continent = "America"
265 WHERE user.id NOT IN (
266     SELECT european_users.id
267     FROM european_users);
268
100% 23:267

```

	Time	Action	Response	Duration / Fetch Time
✓ 269	12:38:38	UPDATE user SET user.continent = "America" WHERE user.i...	1010 row(s) affected Rows matched: 1010 Changed: 1010 Warnings: 0	0.036 sec

- En este caso utilizo **UPDATE** para **añadir America** en la columna user.continent de la tabla user, si los user.id son diferentes a los de european_users.id

```

272 DROP TABLE european_users;
273
100% 27:272

```

	Time	Action	Response	Duration / Fetch Time
✓ 270	12:40:43	DROP TABLE european_users	0 row(s) affected	0.015 sec

- Una vez hecho esto, vuelvo a **eliminar la tabla temporal european_users**

```

276 LOAD DATA INFILE '/private/tmp/CSV_originales/companies.csv'
277 INTO TABLE company
278 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas
279 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
280 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
281 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
282 (
283     @company_id, @company_name, @phone, @email, @country, @website
284 )
285 SET
286     company_id = @company_id,
287     company_name = @company_name,
288     phone = @phone,
289     email = @email,
290     country = @country,
291     website = @website;
292
100% 24:291

```

	Time	Action	Response	Duration / Fetch Time
✓ 271	12:41:23	LOAD DATA INFILE '/private/tmp/CSV_originales/companies....	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.0038 sec

- Cargo los datos del **CSV companies** en la tabla **company** correctamente

```

295 LOAD DATA INFILE '/private/tmp/CSV_originales/credit_cards.csv'
296 INTO TABLE credit_card
297 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas
298 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
299 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
300 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
301 (
302     @id, @user_id, @iban, @pan, @pin, @cvv, @track1, @track2, @expiring_date
303 )
304 SET
305     id = @id,
306     user_id = @user_id,
307     iban = @iban,
308     pan = @pan,
309     pin = @pin,
310     cvv = @cvv,
311     track1 = @track1,
312     track2 = @track2,
313     expiring_date = STR_TO_DATE(@expiring_date, '%m/%d/%y');
314
100% 61:313

```

	Time	Action	Response	Duration / Fetch Time
✓ 272	12:42:02	LOAD DATA INFILE '/private/tmp/CSV_originales/credit_card...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.057 sec

- Cargo los datos del **CSV credit_cards** en la tabla **credit_card** correctamente:
- Utilizando **STR_TO_DATE()** para **convertir la cadena de texto** en una **fecha**, en este caso con;
 - **%m**: Que se utiliza para el mes (01 a 12)
 - **%d**: Que se utiliza para el día del mes (01 a 31), añadiendo la , que tiene a su derecha
 - **%y**: Que se utiliza para el año con 2 dígitos

```

321 LOAD DATA INFILE '/private/tmp/CSV_originales/products.csv'
322 INTO TABLE products
323 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas
324 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
325 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
326 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
327 (
328     @id, @product_name, @price, @colour, @weight, @warehouse_id
329 )
330 SET
331     id = @id,
332     product_name = @product_name,
333     price = @price,
334     colour = @colour,
335     weight = @weight,
336     warehouse_id = @warehouse_id;
337
100% 34:336

```

	Time	Action	Response	Duration / Fetch Time
✗ 273	12:45:04	LOAD DATA INFILE '/private/tmp/CSV_originales/products.c...	Error Code: 1366. Incorrect decimal value: '\$161.11' for column 'price' at row 1	0.0042 sec

- Intento cargar los datos del **CSV products** en la tabla **products**.
- Devuelve un error porque el **símbolo \$** al inicio lo convierte a una cadena de texto y no lo detecta como un decimal válido.

```

341 LOAD DATA INFILE '/private/tmp/CSV_originales/products.csv'
342 INTO TABLE products
343 FIELDS TERMINATED BY ',' -- Indico que los campos están separados por comas
344 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
345 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
346 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
347 (
348     @id, @product_name, @price, @colour, @weight, @warehouse_id
349 )
350 SET
351     id = @id,
352     product_name = @product_name,
353     price = REPLACE(@price, '$', ''),
354     colour = @colour,
355     weight = @weight,
356     warehouse_id = @warehouse_id;
357
100% 34:356

```

	Time	Action	Response	Duration / Fetch Time
✓ 274	12:46:11	LOAD DATA INFILE '/private/tmp/CSV_originales/products.c...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.0053 sec

- Utilizo **REPLACE()** para sustituir **\$** por **" nada"**, para que el valor se convierta en un decimal válido.
- Cargo los datos del **CSV products** en la tabla **products** correctamente.

```
361 LOAD DATA INFILE '/private/tmp/CSV_originales/transactions.csv'
362 INTO TABLE transaction
363 FIELDS TERMINATED BY ';' -- Indico que los campos están separados por comas
364 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
365 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
366 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
367 (
368     @id, @card_id, @business_id, @timestamp, @amount, @declined, @product_ids, @user_id, @lat, @longitude
369 )
370 SET
371     id = @id,
372     card_id = @card_id,
373     business_id = @business_id,
374     timestamp = @timestamp,
375     amount = @amount,
376     declined = @declined,
377     product_ids = @product_ids,
378     user_id = @user_id,
379     lat = @lat,
380     longitude = @longitude;
381
```

	Time	Action	Response	Duration / Fetch Time
▲ 275	12:46:58	LOAD DATA INFILE '/private/tmp/CSV_originales/transaction...	100000 row(s) affected, 1024 warning(s): 1265 Data truncated for column 'lat' at row 1 1265 Data truncated for column...	0.832 sec

- Intento cargar los datos del **CSV transaction** en la tabla **transaction** correctamente.
- Me devuelve un error en el que me dice que el **valor introducido** en **lat** y **longitude** es **demasiado preciso** para el **tipo de dato actual** de la columna, en este caso **DECIMAL(10, 7)** está cortando los decimales.

```
386 DROP TABLE transaction;
387
```

	Time	Action	Response	Duration / Fetch Time
✓ 276	12:48:29	DROP TABLE transaction	0 row(s) affected	0.022 sec

- Elimino la tabla para volver a añadirla después de **modificar** el tipo de dato de **lat** y **longitude** a **DOUBLE**.

```
390 CREATE TABLE IF NOT EXISTS transaction (
391     id VARCHAR(255) PRIMARY KEY,
392     card_id VARCHAR(20),
393     business_id VARCHAR(15),
394     timestamp TIMESTAMP,
395     amount DECIMAL(10, 2),
396     declined TINYINT,
397     product_ids VARCHAR(255),
398     user_id CHAR(10),
399     lat DOUBLE,
400     longitude DOUBLE
401 );
402
```

	Time	Action	Response	Duration / Fetch Time
✓ 277	12:49:13	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(2...	0 row(s) affected	0.016 sec

- Vuelvo a crear la tabla con el **tipo de dato** de **lat** y **longitude** **modificado** a **DOUBLE**.
- El tipo de dato **DOUBLE** utiliza el **doble de memoria** para almacenar el tipo decimal.

```
406 LOAD DATA INFILE '/private/tmp/CSV_originales/transactions.csv'
407 INTO TABLE transaction
408 FIELDS TERMINATED BY ';' -- Indico que los campos están separados por comas
409 ENCLOSED BY '"' -- Indico que debe ignorar las dobles comillas " al leer los campos.
410 LINES TERMINATED BY '\n' -- Indico que cada línea termina con un salto de línea
411 IGNORE 1 ROWS -- Ignoro la primera fila (los encabezados del CSV)
412 (
413     @id, @card_id, @business_id, @timestamp, @amount, @declined, @product_ids, @user_id, @lat, @longitude
414 )
415 SET
416     id = @id,
417     card_id = @card_id,
418     business_id = @business_id,
419     timestamp = @timestamp,
420     amount = @amount,
421     declined = @declined,
422     product_ids = @product_ids,
423     user_id = @user_id,
424     lat = @lat,
425     longitude = @longitude;
```

Time	Action	Response	Duration / Fetch Time
278 12:50:10	LOAD DATA INFILE '/private/tmp/CSV_originales/transaction...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	0.720 sec

- Cargo los datos del **CSV transactions** en la tabla **transaction** correctamente.

Defino relaciones (dejando products para el nivel 3)

```
429 ALTER TABLE transaction
430 ADD FOREIGN KEY (card_id) REFERENCES credit_card(id);
```

Time	Action	Response
287 12:55:52	ALTER TABLE transaction ADD FOREIGN KEY (card_id) REFERENCES cre...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

- Defino la relación entre **transaction** y **credit_card**, donde la **transaction.card_id** es la **FK** y hace referencia a la **PK credit_card.id**.

```
434 ALTER TABLE transaction
435 ADD FOREIGN KEY (business_id) REFERENCES company(company_id);
```

Time	Action	Response
288 12:57:49	ALTER TABLE transaction ADD FOREIGN KEY (business_id) REFERENCE...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

- Defino la relación entre **transaction** y **company**, donde la **transaction.business_id** es la **FK** y hace referencia a la **PK company.company_id**.

```
439 ALTER TABLE transaction
440 ADD FOREIGN KEY (user_id) REFERENCES user(id);
```

Time	Action	Response
302 13:12:18	ALTER TABLE transaction ADD FOREIGN KEY (user_id) REFERENCES user(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

- Defino la relación entre **transaction** y **user**, donde la **transaction.user_id** es la **FK** y hace referencia a la **PK user.id**.

Transformaciones transaction

```
444 ALTER TABLE transaction
445 CHANGE COLUMN business_id company_id VARCHAR(15);
```

Time	Action	Response	Duration / Fetch Time
2 13:18:18	ALTER TABLE transaction CHANGE COLUMN business_id company_id VARCHAR(15)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec

- Cambio el nombre de la columna **transaction.business_id** por **company_id** conservando el tipo de dato **VARCHAR(15)**

```
444 ALTER TABLE transaction
445 CHANGE COLUMN product_ids products_id VARCHAR(20);
446
```

100% 51:445

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 4	13:21:10	ALTER TABLE transaction CHANGE COLUMN product_ids products_id VARCHAR(20)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	0.991 sec

- Cambio el nombre de la columna **transaction.product_ids** por **products_id** conservando el tipo de dato **VARCHAR(20)**

Transformaciones company

```
449 ALTER TABLE company
450 CHANGE COLUMN company_id id VARCHAR(15);
451
```

100% 41:450

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 11	13:25:31	ALTER TABLE company CHANGE COLUMN company_id id VARCHAR(15)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec

- Cambio el nombre de la columna **company.company_id** por **id** conservando el tipo de dato **VARCHAR(15)**

```
454 ALTER TABLE company
455 CHANGE COLUMN company_name name VARCHAR(255);
456
```

100% 46:455

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 12	13:26:30	ALTER TABLE company CHANGE COLUMN company_name name VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.023 sec

- Cambio el nombre de la columna **company.company_name** por **name** conservando el tipo de dato **VARCHAR(255)**

Transformaciones products

```
459 ALTER TABLE products
460 CHANGE COLUMN product_name name VARCHAR(100);
461
```

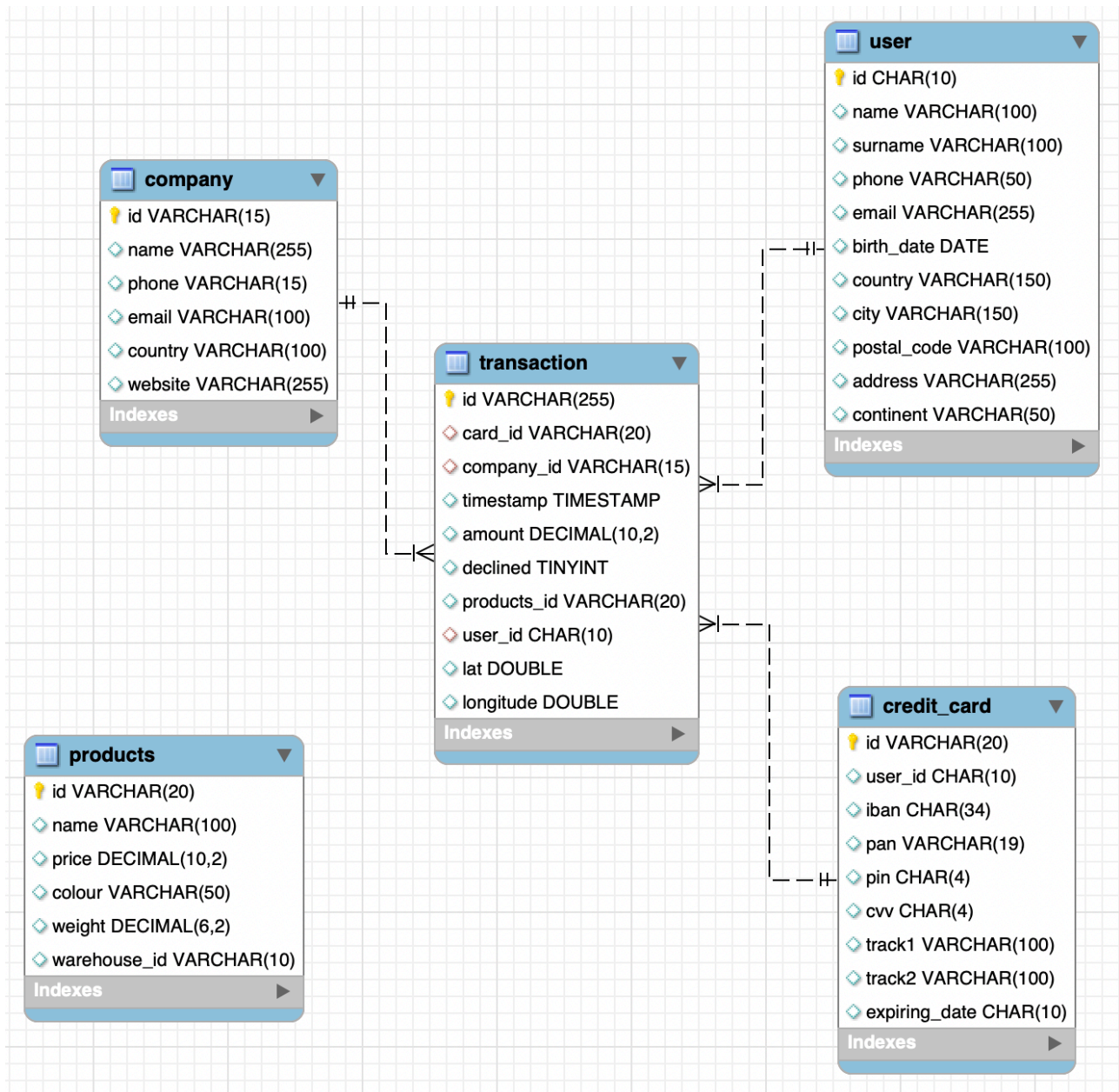
100% 46:460

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 13	13:30:27	ALTER TABLE products CHANGE COLUMN product_name name VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.0080 sec

- Cambio el nombre de la columna **products.product_name** por **name** conservando el tipo de dato **VARCHAR(100)**

DIAGRAMA (sin de definir la relación entre transaction y products)



Relación: Modelo en estrella.

- La relación por parte de (**company**, **credit_card** y **user**) es de (Uno) a (Muchos - Transaction), una empresa, tarjeta de crédito o usuario, puede aparecer en muchas transacciones, pero cada transacción pertenece a una única empresa, un único usuario y a una única tarjeta de crédito.
- **transaction** actúa como la **Tabla de Hechos**, contiene las transacciones realizadas.
- **company** actúa como una **Tabla de Dimensión**, ya que describe cada empresa, (nombre, país, teléfono...)
- **User** y **Credit_Card**, también son tablas de dimensiones que describen las transacciones y añaden contexto al análisis, como quien realiza la transacción, donde y como, todas conectadas a la tabla de hechos **transaction**.

Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
467 SELECT *
468 FROM user
469 WHERE user.id IN (
470     SELECT transaction.user_id
471     FROM transaction
472     GROUP BY transaction.user_id
473     HAVING COUNT(transaction.user_id) >= 80);
```

100% 43:473

Result Grid

id	name	surname	phone	email	birth_date	country	city	postal_code	address	continent
185	Molly	Gilliam	0800 120 8023	donec@outlook.co.uk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.	Europe
289	Dxwgl	Hwcrn	+98-309-8797	dxwgl.hwcrn@example.com	1876-08-20	Germany	Stuttgart	70173	82 Hwcrn Street	Europe
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	1874-05-03	Italy	Genoa	16100	53 Astuw Street	Europe
454	Sfzzoh	Xgvfridx	+58-495-3945	sfzzoh.xgvfridx@example.com	1962-08-28	Poland	Gdansk	80-001	52 Xgvfridx Street	Europe
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

user 11

Action Output

Time	Action	Response	Duration / Fetch Time
13:56:13	SELECT * FROM user WHERE user.id IN (SELECT transaction.user_id FROM transaction G...	4 row(s) returned	0.037 sec / 0.000006...

- En mi consulta de dos niveles;
- **La subquery:** Identifica a los **user_id** que cumplen con el criterio de tener 80 o más transacciones.
- **La query principal:** Usa esos **id** filtrados para mostrar todos los datos de la tabla user.

Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
484 SELECT company.name, credit_card.iban, ROUND(AVG(transaction.amount), 2) AS MitjanaAmountPerIban
485 FROM transaction
486 INNER JOIN credit_card
487 ON transaction.card_id = credit_card.id
488 INNER JOIN company
489 ON transaction.company_id = company.id
490 WHERE transaction.declined = 0
491 AND company.name = "Donec Ltd"
492 GROUP BY company.name, credit_card.iban
493 ORDER BY MitjanaAmountPerIban DESC;
```

100% 36:493 1 error found

Result Grid

name	iban	MitjanaAmountPerIban
Donec Ltd	XX383017813919620199366352	680.69
Donec Ltd	XX637706357397570394973913	680.01
Donec Ltd	XX971393971465292202312259	645.46
Donec Ltd	XX17184711692882375969307	628.89
Donec Ltd	XX225424638818542406223575	608.68
Donec Ltd	XX748890729057195711766071	607.29
Donec Ltd	TN9614563570667381893122	605.41
Donec Ltd	XX481908034037364242591185	605.36
Donec Ltd	XX194675519739256335753508	597.19
Donec Ltd	XX215962766061967195493437	594.26
Donec Ltd	XX449322320826890721001443	591.61
Donec Ltd	XX535185492735704229474237	570.09
Donec Ltd	CH9552373968796160224	566.38
Donec Ltd	XX347605377125637880303131	561.80

Result 21

Action Output

Time	Action	Response	Duration / Fetch Time
13:19:34	SELECT company.name, credit_card.iban, ROUND(AVG(transaction.amount), 2) AS MitjanaAmountPe...	370 row(s) returned	0.012 sec / 0.000063...

- En esta consulta he hecho JOIN entre las 3 tablas para mostrar el nombre de la compañía, el iban de la tarjeta de crédito y un average del amount de cada iban
- Utilizo el **WHERE** para filtrar que solo se base en las transacciones que **no fueron declined**.

NIVELL 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu.

Partint d'aquesta taula respon:

```
513 SELECT transaction.card_id, transaction.declined, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) as num_filas
514 FROM transaction;
```

100% 18:514

Result Grid Filter Rows: Search Export: Fetch rows:

card_id	declined	num_filas
CcS-4857	0	1
CcS-4857	0	2
CcS-4857	0	3
CcS-4857	1	4
CcS-4857	0	5
CcS-4857	0	6
CcS-4857	0	7
CcS-4857	0	8
CcS-4857	0	9
CcS-4857	0	10
CcS-4857	0	11
CcS-4857	0	12
CcS-4857	0	13
CcS-4857	0	14
CcS-4857	0	15
CcS-4857	0	16

Result 17 Read Only

Action Output

Time	Action	Response	Duration / Fetch Time
15...	10:47:48	SELECT transaction.card_id, transaction.declined, ROW_NUMBER() OVER (PARTITION BY card_id OR...	100000 row(s) returned 0.082 sec / 0.060 sec

- Utilizo ROW_NUMBER() sobre la tabla transaction, particionando por card_id para resetear el recuento, ordenando por timestamp descendente, es decir, de más reciente a más antiguo.

```
518 WITH ClasificacionTransaccion AS (
519     SELECT transaction.card_id, transaction.declined, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) as num_filas
520     FROM transaction)
521 SELECT card_id, SUM(declined) AS TotalDeclinedUltimos3
522 FROM ClasificacionTransaccion
523 WHERE num_filas <= 3
524 GROUP BY card_id;
```

100% 18:524

Result Grid Filter Rows: Search Export: Fetch rows:

card_id	TotalDeclinedUltim...
CcS-4857	0
CcS-4858	0
CcS-4859	0
CcS-4860	0
CcS-4861	0
CcS-4862	0
CcS-4863	0
CcS-4864	0
CcS-4865	0
CcS-4866	0
CcS-4867	1
CcS-4868	0
CcS-4869	0
CcS-4870	3
CcS-4871	0
CcS-4872	0

Result 18 Read Only

Action Output

Time	Action	Response	Duration / Fetch Time
15...	10:56:42	WITH ClasificacionTransaccion AS (SELECT transaction.card_id, transaction.declined, ROW_NUMBE...	6000 row(s) returned 0.156 sec / 0.0011 sec

- Utilizo el WITH para crear una tabla temporal denominada ClasificacionTransaccion a la que poder hacer referencia a lo largo de mi consulta.
- Sobre esa tabla temporal ClasificacionTransaccion, muestro los card_id y la SUM() de declined de las últimas 3 transacciones, filtrando el num_filas, previamente ordenado de más reciente a más antiguas, por menor o igual a 3 para que solo muestre las 3 últimas.

```

528 WITH ClasificacionTransaccion AS (
529     SELECT card_id, declined, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) as num_filas
530     FROM transaction)
531 SELECT card_id, CASE
532     WHEN SUM(declined) = 3 THEN 'Inactiu'
533     ELSE 'Actiu'
534 END AS estat
535 FROM ClasificacionTransaccion
536 WHERE num_filas <= 3
537 GROUP BY card_id;

```

100% 18:537

Result Grid

card_id	estat
CcS-4859	Actiu
CcS-4860	Actiu
CcS-4861	Actiu
CcS-4862	Actiu
CcS-4863	Actiu
CcS-4864	Actiu
CcS-4865	Actiu
CcS-4866	Actiu
CcS-4867	Actiu
CcS-4868	Actiu
CcS-4869	Actiu
CcS-4870	Inactiu
CcS-4871	Actiu
CcS-4872	Actiu
CcS-4873	Actiu
CcS-4874	Actiu

Result 19

Action Output

Time	Action	Response	Duration / Fetch Time
15:12:18:03	WITH ClasificacionTransaccion AS (SELECT card_id, declined, ROW_NUMBER() OVER (PARTITION...	6000 row(s) returned	0.158 sec / 0.0021 sec

- Ahora para mostrar el **estado** de cada tarjeta como “**Inactiu**” si tiene 3 declined o “**Actiu**” si no, incluyo el **SUM()** de la query anterior en un **CASE**.

```

543 CREATE TABLE IF NOT EXISTS estado_tarjeta (
544     card_id VARCHAR(20),
545     estat CHAR(10)
546 );
547

```

100% 3:546

Action Output

Time	Action	Response	Duration / Fetch Time
15:12:28:46	CREATE TABLE IF NOT EXISTS estado_tarjeta (card_id VARCHAR(20), estat CHAR(10))	0 row(s) affected	0.014 sec

- Creo la tabla **estado_tarjeta** asignando a **card_id** el mismo tipo de dato **VARCHAR(20)** original que tiene asignado en la tabla **transaction** y en **credit_card** y asigno a **estat** un tipo de dato **CHAR(10)** que es más que suficiente para el valor que tiene que almacenar.

```

550 INSERT INTO estado_tarjeta (card_id, estat)
551 WITH ClasificacionTransaccion AS (
552     SELECT card_id, declined, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) as num_filas
553     FROM transaction)
554 SELECT card_id, CASE
555     WHEN SUM(declined) = 3 THEN 'Inactiu'
556     ELSE 'Actiu'
557 END AS estat
558 FROM ClasificacionTransaccion
559 WHERE num_filas <= 3
560 GROUP BY card_id;
561

```

100% 18:560

Action Output

Time	Action	Response	Duration / Fetch Time
15:12:33:27	INSERT INTO estado_tarjeta (card_id, estat) WITH ClasificacionTransaccion AS (SELECT card_id,...	6000 row(s) affected Records: 6000 Duplicates: 0 Warnings: 0	0.189 sec

- Inserto los datos obtenidos en la nueva tabla **estado_tarjeta**

562	SELECT *
563	FROM estado_tarjeta;
100%	21:563

card_id	estat
CcS-4857	Actiu
CcS-4858	Actiu
CcS-4859	Actiu
CcS-4860	Actiu
CcS-4861	Actiu
CcS-4862	Actiu
CcS-4863	Actiu
CcS-4864	Actiu
CcS-4865	Actiu
CcS-4866	Actiu
CcS-4867	Actiu
CcS-4868	Actiu
CcS-4869	Actiu
CcS-4870	Inactiu
CcS-4871	Actiu
CcS-4872	Actiu

Time	Action	Response	Duration / Fetch Time
15...	12:34:42	SELECT * FROM estado_tarjeta	5000 row(s) returned 0.0014 sec / 0.0032...

- Muestro todos los datos de la nueva tabla **estado_tarjeta**

Exercici 1

Quantes targetes estan actives?

560	SELECT COUNT(estado_tarjeta.estat) AS NumTargetesActives
561	FROM estado_tarjeta
562	WHERE estat = "Actiu";
100%	23:562

NumTargetesActives
4995

Time	Action	Response	Duration / Fetch Time
15...	12:44:32	SELECT COUNT(estado_tarjeta.estat) AS NumTargetesActives FROM estado_tarjeta WHERE estat = "Actiu";	1 row(s) returned 0.0080 sec / 0.0000...

- Hago un **COUNT()** del estado de las tarjetas en la nueva tabla **filtrando** por **Actiu**

Nivell 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

568	CREATE TABLE IF NOT EXISTS ProductosPorTransaccion (
569	transaction_id VARCHAR(255),
570	product_id VARCHAR(20),
571	PRIMARY KEY (transaction_id, product_id),
572	FOREIGN KEY (transaction_id) REFERENCES transaction(id),
573	FOREIGN KEY (product_id) REFERENCES products(id)
574	);
575	
100%	3:574

Time	Action	Response	Duration / Fetch Time
15...	12:52:31	CREATE TABLE IF NOT EXISTS ProductosPorTransaccion (transaction_id VARCHAR(255), produ...	0 row(s) affected 0.018 sec

- Creo una nueva tabla llamada ProductosPorTransaccion, con la siguiente estructura;
- **transaction_id** con el tipo de dato **VARCHAR(255)** siendo el mismo tipo de dato asignado en la tabla **transaction**.
- **product_id** con el tipo de dato **VARCHAR(20)** siendo el mismo tipo de dato asignado en la tabla **products**.
- Asigno ambas columnas **transaction_id** y **product_id** como **Primary Key**
- Defino la **Foreign Key** de **transaction_id** que hace referencia a **transaction(id)**
- Defino la **Foreign Key** de **product_id** que hace referencia a **products(id)**

```

576 ALTER TABLE transaction
577 MODIFY COLUMN products_id JSON;
578
100% 32:577

```

Action Output

	Time	Action	Response	Duration / Fetch Time
6	09:19:02	ALTER TABLE transaction MODIFY COLUMN products_id JSON	Error Code: 3140. Invalid JSON text: "The document root must not be followed by other values." at position 2 in value...	0.016 sec

- Para poder **extraer** cada **producto** por **separado** de la columna **products_id** en **transaction**, intento cambiar el tipo de dato a JSON.
- Me devuelve error porque los datos dentro de esa columna **"12, 34, 56"** no parecen **JSON** → **["12", "34", "56"]**, y **MYSQL** no **permite** cambiar directamente una **cadena de texto** a **objeto JSON**

```

579 ALTER TABLE transaction
580 ADD COLUMN products_id_json JSON;
581
100% 34:580

```

Action Output

	Time	Action	Response	Duration / Fetch Time
7	09:26:21	ALTER TABLE transaction ADD COLUMN products_id_json JS...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.043 sec

- Decido añadir una columna temporal nueva, para posteriormente transformar esos datos.

```

582 UPDATE transaction
583 SET products_id_json = CAST(CONCAT('[', products_id, ']') AS JSON);
584
100% 68:583

```

Action Output

	Time	Action	Response	Duration / Fetch Time
9	09:32:42	UPDATE transaction SET products_id_json = CAST(CONCAT(...	100000 row(s) affected Rows matched: 100000 Changed: 100000 Warnings: 0	1.879 sec

- Añado los datos de la columna original **products_id** a la nueva columna **products_id_json** con las siguientes modificaciones:
- **CONCAT("[", ..., ""])**: Añade **"["** al principio y **"]"** al final. El resultado es **["12", "34", "56"]**.
- **CAST(... AS JSON)**: Le dice a **MYSQL**: "Esto que parece texto es en realidad un objeto JSON, guárdalo como tal".

```

585 SELECT products_id, products_id_json
586 FROM transaction;
587
100% 18:586

```

Result Grid

products_id	products_id_json
16, 26, 97, 87	[16, 26, 97, 87]
66, 69, 87	[66, 69, 87]
30, 11, 16, 81	[30, 11, 16, 81]
72	[72]
18	[18]
35, 33, 19	[35, 33, 19]
93, 55, 28, 91	[93, 55, 28, 91]
55, 8, 72	[55, 8, 72]
46, 56, 73	[46, 56, 73]
6, 89, 19	[6, 89, 19]
58, 61, 55	[58, 61, 55]
62	[62]
36	[36]
8, 91	[8, 91]
76, 19, 89	[76, 19, 89]
3	[3]
54, 17	[54, 17]
88, 4	[88, 4]
52, 3, 91, 75	[52, 3, 91, 75]
64	[64]
85	[85]
71	[71]

transaction 1

Action Output

	Time	Action	Response	Duration / Fetch Time
11	09:45:54	SELECT products_id, products_id_json FROM transaction	100000 row(s) returned	0.0020 sec / 0.055 sec

- Muestro ambas columnas con una **SELECT**.
- Pero me doy cuenta de que el resultado no es **["12", "34", "56"]** sino **[12, 34, 56]** y entiendo que como tienen un **espacio después de la coma**, debo cambiar el formato para que **JSON** no se confunda al leer los datos.

```

588 UPDATE transaction
589 SET products_id_json = CAST(CONCAT('[', REPLACE(products_id, ',', '","'), ']') AS JSON);
590
100% 92:589

```

Action Output

	Time	Action	Response	Duration / Fetch Time
12	09:52:15	UPDATE transaction SET products_id_json = CAST(CONCAT(...	100000 row(s) affected Rows matched: 100000 Changed: 100000 Warnings: 0	1.016 sec

- Añado al código anterior un **REPLACE()** para solucionar lo del espacio después de la coma
- **REPLACE(products_id, ',', '","')**: Cambia cada **" , "** por **" , "**. Así, **12, 34** se convierte en **12","34**.

591 SELECT products_id, products_id_json
592 FROM transaction;
593

100% 18:592

Result Grid Filter Rows: Search Export: Fetch rows:

products_id	products_id_json
16, 26, 97, 87	[{"16", "26", "97", "87"}]
66, 69, 87	[{"66", "69", "87"}]
30, 11, 16, 81	[{"30", "11", "16", "81"}]
72	[{"72"}]
18	[{"18"}]
35, 33, 19	[{"35", "33", "19"}]
93, 55, 28, 91	[{"93", "55", "28", "91"}]
55, 8, 72	[{"55", "8", "72"}]
46, 56, 73	[{"46", "56", "73"}]
6, 89, 19	[{"6", "89", "19"}]
58, 61, 55	[{"58", "61", "55"}]
62	[{"62"}]
36	[{"36"}]
8, 91	[{"8", "91"}]
76, 19, 89	[{"76", "19", "89"}]
3	[{"3"}]
54, 17	[{"54", "17"}]
88, 4	[{"88", "4"}]
52, 9, 91, 75	[{"52", "9", "91", "75"}]
64	[{"64"}]
85	[{"85"}]
71	[{"71"}]

transaction 2 Read Only

Action Output

Time	Action	Response	Duration / Fetch Time
13 09:54:57	SELECT products_id, products_id_json FROM transaction	100000 row(s) returned	0.0017 sec / 0.055 sec

- Vuelvo a mostrar las dos columnas con una **SELECT** y esta vez todo ha salido correctamente.
- Ahora para insertar los datos en la tabla de unión **ProductosPorTransaccion**, necesito desglosar ese arreglo **JSON** para que cada **product_id** se convierta en una fila individual.

615 SELECT transaction.id AS transaction_id, tabla_json.product_id
616 FROM transaction, JSON_TABLE(
617 transaction.products_id_json, '\$[*]' COLUMNS (product_id VARCHAR(20) PATH '\$')) AS tabla_json;
618

100% 96:617

Result Grid Filter Rows: Search Export: Fetch rows:

transaction_id	product_id
00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
00043A49-2949-494B-A5DD-A5BAE3BB19DD	97
00043A49-2949-494B-A5DD-A5BAE3BB19DD	87
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	66
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	69
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	87
00045D6B-ED2E-4F2F-8186-CEE074D875D0	30
00045D6B-ED2E-4F2F-8186-CEE074D875D0	11
00045D6B-ED2E-4F2F-8186-CEE074D875D0	16
00045D6B-ED2E-4F2F-8186-CEE074D875D0	81
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	72
00051AA4-9CBE-4268-B070-C38062A1B3E2	18
0008A312-EDFE-4A4F-BC99-E9C82EC3CA4D	35
0008A312-EDFE-4A4F-BC99-E9C82EC3CA4D	33
0008A312-EDFE-4A4F-BC99-E9C82EC3CA4D	19
0009A151-9BCF-4E31-9053-A468FF77FAAB	93
0009A151-9BCF-4E31-9053-A468FF77FAAB	55

Result 7 Read Only

Action Output

Time	Action	Response	Duration / Fetch Time
22 10:18:45	SELECT transaction.id AS transaction_id, tabla_json.product...	253391 row(s) returned	0.0014 sec / 0.130 sec

- Usaré la función **JSON_TABLE**, que actúa como un "traductor" que **convierte** los **datos** de un **JSON** en una **tabla temporal con filas y columnas**.
- **[\$[*]]**: Le dice a MySQL que recorra **todos** los elementos del arreglo.
- **COLUMNS**: Define cómo se llamará la columna.
- **PATH '\$'**: Indica que coja el valor directo del elemento actual del JSON.

625 INSERT INTO ProductosPorTransaccion (transaction_id, product_id)
626 SELECT transaction.id AS transaction_id, json.product_id
627 FROM transaction, JSON_TABLE(
628 transaction.products_id_json, '\$[*]' COLUMNS (product_id VARCHAR(20) PATH '\$')) AS json;
629

100% 90:628

Action Output

Time	Action	Response	Duration / Fetch Time
30 10:22:19	INSERT INTO ProductosPorTransaccion (transaction_id, prod...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	2.184 sec

- Inserto los **datos** obtenidos en la consulta anterior, a la nueva tabla de unión **ProductosPorTransaccion**.

632SELECT *
633FROM ProductosPorTransaccion;
634
100%30:633

Result Grid

Filter Rows: Search

Edit: Export/Import: Fetch rows:

transaction_id	product_id
001A60EA-DC9C-4E5A-9460-6628B100E7E1	1
0032F0BB-BBE6-4AA5-B5EE-EEAD533C0C48	1
00342381-503D-422D-85AB-F2D4FFAD4C7	1
004C0A80-E537-46D8-BE44-343D2176DF15	1
004D1DB5-B2CB-4460-98B6-31C42CA98E5F	1
0062599C-0A1F-4405-AD55-45E20043B551	1
007B1297-C3ED-4966-802C-68FE106AD25C	1
007EA15D-AA1F-4FB3-9483-7D055E99CEC0	1
009679D1-1849-47C1-98E8-8F6C0EA5DF32	1
0096A25B-BDAD-47DE-85AD-BA3741BE7736	1
00C61197-05FF-4ECD-A523-E0D6E2E3DDDD	1
00EB00C8-F09E-4662-88A4-6D16DF32F0E5	1
00EA97E4-9FA1-42EC-85BC-ECBE44AAE558	1
00EAF316-6667-4A1F-8CA6-53EDF7EDDF6E	1
00F45CC2-960C-42EF-B4AD-0E4DCC0580BB	1
00FD4038-FA8D-4582-BB5F-9C1059E5C340	1
011692F7-7669-48AF-B492-2A7466AD977A	1
0123BF67-2544-4182-94D4-F2674F37A6BA	1

ProductosPorTransaccion 8

Apply

Result Grid

Form Editor

Field Types

Query Stats

Action Output

	Time	Action	Response	Duration / Fetch Time
38	10:27:24	SELECT * FROM ProductosPorTransaccion	253391 row(s) returned	0.0011 sec / 0.059 sec

- Muestro todos los datos de la tabla **ProductosPorTransaccion** y veo que se han cargado correctamente donde **cada fila** es una transacción por producto.

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

642 ■ SELECT products.id AS idProducte, products.name NomProducte, COUNT(ProductosPorTransaccion.product_id) AS NumVegadesVenut

643 FROM ProductosPorTransaccion

644 INNER JOIN products

645 ON ProductosPorTransaccion.product_id = products.id

646 INNER JOIN transaction

647 ON transaction.id = ProductosPorTransaccion.transaction_id

648 WHERE transaction.declined = 0

649 GROUP BY idProducte, NomProducte

650 ORDER BY idProducte;

100% ▾ 21:650

Result Grid

Filter Rows: 🔍 Search

Export: 📄

idProducte	NomProducte	NumVegadesVenut
1	Direwolf Stannis	2467
10	Karstark Dome	2561
100	south duel	2507
11	Karstark Dome	2559
12	duel Direwolf	2548
13	palpatine chewbacca	2555
14	Direwolf	2489
15	Stannis warden	2524
16	the duel warden	2502
17	skywalker ewok sith	2523
18	Karstark warden	2542
19	dooku solo	2446
2	Tarly Stark	2562
20	warden Karstark	2492

Result Grid

Form Editor

Field Types

Result 12

Read Only

Action Output ▾

	Time	Action	Response	Duration / Fetch Time
✓ 94	12:56:09	SELECT products.id AS idProducte, products.name NomProducte, COUNT(ProductosPorTransaccion...	100 row(s) returned	0.554 sec / 0.000062...

- Hago **JOIN** entre las tablas **products**, **ProductosPorTransaccion** y **transaction** para mostrar el **ID** y **nombre** desde la tabla **products**, el **recuento** de las veces que se ha vendido cada producto desde la tabla **ProductosPorTransaccion**, y poder filtrar desde la tabla **transaction** por **declined**.
- Utilizo el **WHERE** para filtrar que solo se base en las transacciones que no fueron declinedas.
- En el **GROUP BY** añado el **ID** de la tabla **products**, porque hay varios nombres de producto repetidos con diferente **ID** y así no se pierde ningún producto en mi query.
- Aunque no lo pide el enunciado, si quisiera ordenar por **idProducte** me encontraría por el siguiente problema.
- Al ordenarlo por **ID** me daría cuenta de que no me lo está haciendo bien, en vez de ordenar por orden numérico, está haciéndolo de manera alfabetico ya que lo interpreta como **VARCHAR** y no como un **INT**.


```
652 SELECT CAST(products.id AS UNSIGNED) AS idProducte, products.name NomProducte, COUNT(ProductosPorTransaccion.product_id) AS NumVegadesVenut
653 FROM ProductosPorTransaccion
654 INNER JOIN products
655 ON ProductosPorTransaccion.product_id = products.id
656 INNER JOIN transaction
657 ON transaction.id = ProductosPorTransaccion.transaction_id
658 WHERE transaction.declined = 0
659 GROUP BY idProducte, NomProducte
660 ORDER BY idProducte;
```

100% 21:660

Result Grid Filter Rows: Search Export:

idProducte	NomProducte	NumVegadesVenut
1	Direwolf Stannis	2467
2	Tarly Stark	2562
3	duel tourney Lannister	2520
4	warden south duel	2573
5	skywalker ewok	2543
6	dooku solo	2487
7	north of Casterly	2555
8	Winterfell	2495
9	Winterfell	2508
10	Karstark Dome	2561
11	Karstark Dome	2559
12	duel Direwolf	2548
13	palpatine chewbacca	2555
14	Direwolf	2489

Result 13 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 95	13:05:35	SELECT CAST(products.id AS UNSIGNED) AS idProducte, products.name NomProducte, COUNT(Pro...	100 row(s) returned	0.563 sec / 0.00004...

- En este punto usaría un **CAST() AS UNSIGNED** para solucionarlo, escojo **UNSIGNED** por la naturaleza de los **IDs** que suelen ser números enteros y positivos, así podría ordenarlo correctamente.